

Neuro-Symbolic AI Approaches to Enhance Deep Neural Networks
with Logical Reasoning and Knowledge Integration

by

Zhun Yang

A Dissertation Presented in Partial Fulfillment
of the Requirements for the Degree
Doctor of Philosophy

Approved April 2023 by the
Graduate Supervisory Committee:

Joohyung Lee, Chair
Chitta Baral
Baoxin Li
Yezhou Yang

ARIZONA STATE UNIVERSITY

August 2023

ABSTRACT

One of the challenges in Artificial Intelligence (AI) is to integrate fast, automatic, and intuitive System-1 thinking with slow, deliberate, and logical System-2 thinking. While deep learning approaches excel at perception tasks for System-1, their reasoning capabilities for System-2 are limited. Besides, deep learning approaches are usually data-hungry, hard to make use of explicit knowledge, and struggling with interpretability and justification. This dissertation presents three neuro-symbolic AI approaches that integrate neural networks (NNs) with symbolic AI methods to address these issues.

The first approach presented in this dissertation is NeurASP, which combines NNs with Answer Set Programming (ASP), a logic programming formalism. NeurASP provides an effective way to integrate sub-symbolic and symbolic computation by treating NN outputs as probability distributions over atomic facts in ASP. The explicit knowledge encoded in ASP corrects mistakes in NN outputs and allows for better training with less data.

To avoid NeurASP's bottleneck in symbolic computation, this dissertation presents a Constraint Loss via Straight-Through Estimators (CL-STE). CL-STE provides a systematic way to compile discrete logical constraints into a loss function over discretized NN outputs and scales significantly better than state-of-the-art neuro-symbolic methods. This dissertation also presents a finding when CL-STE was applied to Transformers. Transformers can be extended with recurrence to enhance its power for multi-step reasoning. Such Recurrent Transformer can straightforwardly be applied to visual constraint reasoning problems while successfully addressing the symbol grounding problem.

Lastly, this dissertation addresses the limitation of pre-trained Large Language Models (LLMs) on multi-step logical reasoning problems with a dual-process neuro-symbolic reasoning system called LLM+ASP, where an LLM (e.g., GPT-3) serves as a highly effective few-shot semantic parser that turns natural language sentences into a logical form that can be used as input to ASP. LLM+ASP achieves state-of-the-art performance on several tex-

tual reasoning benchmarks and can handle robot planning tasks that an LLM alone fails to solve.

PREVIEW

*To my beloved wife, parents, and the little angel who will soon join us, for their
unwavering support and encouragement every step of the way.*

ACKNOWLEDGMENTS

I am deeply grateful to many individuals who have supported me throughout my academic journey.

Firstly, I would like to express my sincere appreciation to my advisor, Joohyung Lee, for his unwavering guidance and support throughout my academic journey. I first met Joohyung when I was an MS student in his Artificial Intelligence (AI) course, which sparked my interest in AI and inspired me to pursue research in the field. Since then, he has been a remarkable mentor for my MS thesis and Ph.D. dissertation, providing countless insightful suggestions and feedback that have been instrumental in shaping my research. I am also grateful for the opportunities he has provided me to present my research at top-tier conferences, to help organize an international conference (KR 2018), and to serve as a judge representing AAAI in Intel Science and Engineering Fair 2019. I am fortunate to have had such a patient, dedicated, and generous advisor.

I would also like to thank the members of the Automated Reasoning Group (ARG) at ASU. Manjula Malaizarasan, Nikhil Loney, Samidh Talsania, Anish Pradhan, Yi Wang, Jiaxuan Pang, Jinyung Hong, Man Luo, and Adam Ishay have all been instrumental in shaping my work and helping me grow as a good researcher and programmer. Their feedback, mentorship, and support have been invaluable. I would like to offer special thanks to Yi Wang and Adam Ishay for their exceptional assistance, collaboration, and feedback throughout my academic journey.

I would also like to extend my sincere gratitude to my committee members, Chitta Baral, Baoxin Li, and Yezhou Yang, for their invaluable support and guidance throughout my research. Chitta Baral, in particular, has been an incredible mentor and advocate for my work. He has invited me to present my research in his lab and class and has provided insightful suggestions. Baoxin Li and Yezhou Yang have also been incredibly supportive and have provided valuable feedback and suggestions for both research and career path.

I feel fortunate to have had such a knowledgeable and supportive committee, and their contributions have been invaluable to my academic journey.

Finally, I would like to express my deepest gratitude to my wife, Jing Zhou, for her unwavering support, encouragement, and understanding throughout my entire Ph.D. journey. Her love and presence have been a constant source of strength and inspiration. I would also like to thank my parents, Fenlin Xia and Chunping Yang, for their unconditional love, guidance, and inspiration. They have always believed in me, and their unwavering faith in my abilities has motivated me to study abroad and pursue my dreams. And lastly, I want to thank my little angel, who is on her way to join our family, for bringing added blessings and inspiration to my life.

My work in this thesis was partially supported by the National Science Foundation under Grants IIS-1319794, IIS-1526301, IIS-1815337, and IIS-2006747.

TABLE OF CONTENTS

	Page
LIST OF TABLES	x
LIST OF FIGURES	xii
 CHAPTER	
1 INTRODUCTION	1
2 BACKGROUND	6
2.1 Neuro-Symbolic Methods	6
2.2 Answer Set Programs	8
3 NEURASP	11
3.1 Syntax	11
3.2 Semantics	14
3.3 Inference with <code>NeurASP</code>	16
3.3.1 Commonsense Reasoning about Image	16
3.3.2 Solving Sudoku Puzzle in Image	18
3.4 Learning in <code>NeurASP</code>	22
3.4.1 Learning Digit Classification from Addition	23
3.4.2 Learning How to Solve Sudoku	25
3.4.3 Learning Shortest Path (SP)	26
3.5 Extend <code>NeurASP</code> with Probabilistic Rules	29
3.5.1 Multi-Valued Probabilistic Programs	29
3.5.2 Define <code>NeurASP</code> on a Translation to MVPP	32
3.6 Detailed Description of Learning Algorithms for <code>NeurASP</code>	35
3.7 Proof of Proposition 1	40
4 CL-STE	43
4.1 STE and Trainable Gate Function	43

CHAPTER	Page
4.2 Enforcing Logical Constraints using STE	46
4.2.1 Encoding CNF as a Loss Function Using STE.....	46
4.2.2 Properties of Constraint Loss and Its Gradients	51
4.3 Evaluation of CL-STE	55
4.3.1 mnistAdd Revisited	56
4.3.2 Benchmarks from (Tsamoura <i>et al.</i> , 2021)	57
4.3.3 CNN + Constraint Loss for Sudoku	59
4.3.4 GNN + Constraint Loss for Sudoku	60
4.3.5 Learning to Solve the Shortest Path Problem	63
4.3.6 Semi-Supervised Learning for MNIST and FASHION Dataset ..	66
4.3.7 Ablation Study with GNN and Constraint Loss for Sudoku	68
4.3.8 Discussion.....	70
4.4 CNF and Total Loss	72
4.4.1 mnistAdd2	72
4.4.2 mnistAdd using $b(x)$ and iSTE.....	73
4.4.3 add2x2	74
4.4.4 apply2x2	75
4.4.5 member(n)	77
4.4.6 Sudoku.....	78
4.5 Proofs.....	80
4.5.1 Proof of Proposition 2	80
4.5.2 Proof of Proposition 3	82
4.5.3 Proof of Proposition 4	88
4.5.4 Proof of Proposition 5	96

CHAPTER	Page
5 RECURRENT TRANSFORMER	105
5.1 CSP	107
5.2 Model Design	108
5.3 Training Objective	111
5.4 Evaluation	112
5.4.1 Textual and Visual Sudoku	112
5.4.2 More Domains	115
5.4.3 Ablation Study	117
5.5 Cardinality Constraint	122
5.5.1 Injecting General Cardinality Constraints via STE	122
5.5.2 Effect of Combinations of Constraints	124
5.5.3 Computation Size	130
6 LLM AND ASP	132
6.1 Method: LLM+ASP	132
6.1.1 Prompts for Fact Extraction	133
6.1.2 Knowledge Modules	135
6.2 Evaluation	136
6.2.1 bAbI	137
6.2.2 StepGame	138
6.2.3 CLUTRR	141
6.2.4 gSCAN	143
6.2.5 Robot Planning	145
6.2.6 Findings	146
6.3 GPT-3 Errors in Semantic Parsing	148

CHAPTER	Page
6.3.1 Argument Misorder	148
6.3.2 Wrong Relation	149
6.3.3 Ambiguous or Incorrect Co-reference.....	150
6.3.4 Anonymous Argument.....	150
6.3.5 Missed to Generate Some Atoms	151
6.4 Dataset Errors.....	151
6.4.1 bAbI.....	151
6.4.2 CLUTRR.....	154
6.5 Prompts for Semantic Parsing	155
6.6 ASP Knowledge Modules	157
7 CONCLUSION	159
7.1 Summary of Contributions	160
7.2 Future Directions.....	162
REFERENCES	165

LIST OF TABLES

Table	Page
3.1 Sudoku: Accuracy on Test Data	20
3.2 Shortest Path: Accuracy on Test Data: Columns Denote MLPs Trained with Different Rules; Each Row Represents the Percentage of Predictions that Satisfy the Constraints	28
4.1 Experiments on mnistAdd	57
4.2 Comparison between CL-STE and Other Approaches: The Numbers in Parentheses Are the Times Spent by NeuroLog to Generate All Abductive Proofs.	58
4.3 CNN, NeurASP, and CL-STE on Park 70k Sudoku Dataset (80%/20% split) w/ and w/o Inference Trick	60
4.4 SATNet vs. CNN+CL-STE	61
4.5 Accuracy on MNIST & FASHION dataset	67
4.6 Clauses in the CNF for member(3) Problem	77
5.1 Whole Board Accuracy on Different Sudoku Datasets. RRN-hardest Con- sists of a Copy of the RRN Training Set, While the Testing Set Consists of Only the Hardest Puzzles with 17 Given Digits in the RRN Test Set. We Compare with 3 Baselines: RRN (Palm <i>et al.</i> , 2018), SATNet (Wang <i>et al.</i> , 2019), And SATNet* (Topan <i>et al.</i> , 2021).	114
5.2 Parameter Values And Counts for L1R32H4 Model for Textual Sudoku	115
5.3 Whole-board Test Accuracy on SATNet (9k/1k)	121
5.4 Whole-board Test Accuracy on Palm (9k/1k)	121
5.5 Cell Test Accuracy on Palm (3k/1k)	121

5.6	Effect of Adding Constraint Losses $\mathcal{L}_{attention}$ (att) And $\mathcal{L}_{Sudoku}$ (sud) to the Baseline Loss $\mathcal{L}_{base}$ When Training the Same L1R32 Model on 9k RRN or RRN-V Training Data.	125
5.7	Effect of Adding Constraint Loss $\mathcal{L}_{constraint}$ And x Thousand Unlabeled Data (Denoted by xkU) When Training the Same L1R32 Model on 4k Labeled RRN or RRN-V Training Data (Denoted by 4kL).....	125
5.8	Constraint Accuracy on SP4 Test Data for the Shortest Path Problem	129
5.9	Computation Size of Different Losses ($R = 32$, $NumAtom = 729$, $NumClause = 8991$)	131
6.1	Test Accuracy on 20 Tasks in bAbI Data	137
6.2	Test Accuracy on the StepGame Test Dataset, Where (c1), (d2), and (d3) Denote text-curie-001, text-davinci-002, and text-davinci-003 Models, Respectively	139
6.3	Test Accuracy on 4 Categories in CLUTRR 1.0 and CLUTRR 1.3 Datasets .	141
6.4	Test Accuracy on CLUTRR-S Dataset	142
6.5	Test Accuracy on the gSCAN Dataset.....	144
6.6	Test Accuracy on the Pick&Place Dataset. (d3) Denotes the text-davinci-003 Model.....	146
6.7	Knowledge Modules Used for Each of the Tasks. Note That DEC Axioms, Action, and Location Modules are Used in at Least Two Datasets.	158

LIST OF FIGURES

Figure		Page
3.1	Reasoning about Relations among Perceived Objects	16
3.2	NeurASP Gradient Propagation	24
3.3	NeurASP v.s. DeepProbLog	24
4.1	Trainable Gate Function $\tilde{b}^K(x)$ When $g(x) = 1$	45
4.2	Architecture That Overlays Constraint Loss	47
4.3	Comparison on mnistAdd	56
4.4	Test Accuracy on the Same Randomly Sampled 1k Data from Palm Sudoku Dataset When Trained with RRN(+STE) with 30k to 60k [L]abeled/[U]nabeled Data	62
4.5	Semi-supervised Learning with RRN+STE on Sudoku Using Only 10k La- beled Data and Varying Numbers of Unlabeled Data from Palm Dataset for Training and Using the Same Randomly Sampled 1k Data for Testing	63
4.6	MLP+CL-STE on Shortest Path Problem	65
4.7	Acc with 30k Dataset under Different Losses	69
4.8	Acc with 30k Dataset under Different Losses in L_{cl}	70
4.9	Acc with 60k Dataset under Different Losses	71
5.1	(a) Transformer Encoder. (b) Recurrent Transformer	110
5.2	Recurrent Transformer for Visual Sudoku Problem.	110
5.3	(left) Running Average of the Test Accuracy for Every 10 Epochs of a Re- current Transformer with Different L And R Trained on the Same 8k RRN Data. (right) Test Accuracy as a Function of the Number T of Recurrences When Testing on Different Difficulty Puzzles in RRN Dataset, Using the Same L1R32 Model Trained on 180k RRN Data.	117

Figure	Page
5.4 (left) Heatmaps of the Learned 81x81 Attention Matrices in the L1R32 Recurrent Transformer with Varying Numbers of Heads. (right) Test Accuracy VS. Epochs for These Models.	118
5.5 (left) The Whole Board Accuracy And Solution Accuracy of the L1R32 Model Trained on Grounded or Ungrounded RRN-V Dataset (9k/1k). (right) The Givens Cell Accuracy of the Same Models.	120
5.6 Effect of adding constraint loss $\mathcal{L}_{constraint}$ and x thousand Unlabeled data (denoted by xkU) when training the same L1R32 model on 4k Labeled RRN training data (denoted by 4kL).	126
5.7 Effect of adding constraint loss $\mathcal{L}_{constraint}$ and x thousand Unlabeled data (denoted by xkU) when training the same L1R32 model on 4k Labeled RRN-V training data (denoted by 4kL).	126
5.8 Recurrent Transformer for Shortest Path.	128
6.1 The LLM+ASP Pipeline for the StepGame Dataset.	133
6.2 The Knowledge Modules at the Bottom Are Used in Each Task on the Top..	136
6.3 The LLM+ASP Pipeline for Pick&Place	145
6.4 A Simple Plan Predicted by LLM+ASP in the Pick&Place Domain.	147

Chapter 1

INTRODUCTION

System 1 and system 2 thinking (Kahneman, 2011) are two distinct modes of cognitive processing: system 1 thinking is fast, automatic, and intuitive while system 2 thinking is slow, deliberate, and logical. These years, system 1 thinking is widely modeled by deep learning approaches, which excel at perception tasks such as language (Vaswani *et al.*, 2017; Zhang *et al.*, 2020; Helwe *et al.*, 2021; Li *et al.*, 2020) and vision (Dosovitskiy *et al.*, 2020; Gabeur *et al.*, 2020) with unprecedented success.

Although there has been some success to implement system 2 thinking in deep neural networks (Šourek *et al.*, 2015; Rocktäschel and Riedel, 2017; Donadello *et al.*, 2017; Kazemi and Poole, 2018; Cohen *et al.*, 2018; Palm *et al.*, 2018; Lin *et al.*, 2019), the reasoning supported in neural networks is still considered shallow (Helwe *et al.*, 2021). For example, while large language models (LLMs) are considered one of the most powerful deep neural network models for various tasks (Brown *et al.*, 2020), Nye *et al.* (2021) noted that LLMs work well for system 1 intuitive thinking but not for system 2 logical thinking. Creswell *et al.* (2022) also asserted that LLMs tend to perform poorly on multi-step logical reasoning problems. On the other hand, the latter subject has been well-studied in the area of symbolic AI, which allows for convenient representation of knowledge (Lifschitz, 2008a; Brewka *et al.*, 2011b; Lee and Wang, 2016) and complex reasoning on both certainty and uncertainty (Katzouris and Artikis, 2020; Verreet *et al.*, 2022; Lee and Yang, 2023).

Symbolic AI does not incorporate high-dimensional vector space and state-of-the-art models for perception tasks as handled in deep neural networks, which limits the applicability of symbolic AI in many practical applications. On the other hand, deep neural net-

works are usually data-hungry and hard to make use of explicit (commonsense or expert) knowledge. They also lack explainability and justification since rich amount of information are stored in vector space and hard to interpret.

Since the strengths of deep neural networks and symbolic AI approaches are complementary, neuro-symbolic AI (Besold *et al.*, 2017; Mao *et al.*, 2019; De Raedt *et al.*, 2019; Garcez *et al.*, 2019) became an active research area with the hopes of a best-of-both worlds scenario. There is no consensus on such combination and various approaches have been proposed with different strengths.

To keep sound reasoning, some recent works in neuro-symbolic AI (Manhaeve *et al.*, 2018; Tsamoura *et al.*, 2021) associate continuous parameters in neural networks (NNs) with logic languages so that logical reasoning applied to NN outputs produces “semantic loss” (Xu *et al.*, 2018). However, these logic languages do not allow many flexible Knowledge Representation (KR) constructs, such as recursive definition, cardinality constraints, and weak constraints, which are supported by answer set programming (ASP) (Lifschitz, 2008b; Brewka *et al.*, 2011a) for convenient representation of complex knowledge. To fill in the gap, we proposed a neuro-symbolic formalism *NeurASP* (Yang *et al.*, 2020), which is a simple extension of ASP where NN outputs are treated as probability distributions over atomic facts in answer set programs. *NeurASP* provides a convenient way to enforce structured knowledge in NNs. We defined the probability of each “intended model” in *NeurASP* using neural network outputs and proved the mathematical properties of its gradients. We empirically showed that *NeurASP* can improve a neural network’s perception result by applying symbolic reasoning in ASP and train a neural network better with ASP rules so that a neural network not only learns from implicit correlations from the data but also from the explicit complex semantic constraints expressed by the rules. Remarkably, the same CNN for Sudoku (Park, 2018) trained by *NeurASP* achieves 66.5% accuracy with 70k unlabeled data while the baseline only achieves 23.3% accuracy with

fully-labeled data.

On the other hand, the symbolic computation is often the bottleneck of the above methods, limits the computation to CPUs only, and restricts their domain of usage to relatively small scale. To allow more time-efficient learning and larger domains, we design a semantic regularization method CL-STE (Yang *et al.*, 2022), which provides a systematic way to represent discrete logical constraints as a regularization function over discretized NN output. To make a discretizing function meaningfully differentiable, we turn to the idea of straight-through estimators (STE) (Courbariaux *et al.*, 2015), which were originally introduced to train binary neural networks — neural networks with binary weights and activation at run-time. The main idea of STE is to use a binarization function in forward propagation while its gradient, which is zero almost everywhere, is replaced by the gradient of a different function in backward propagation. We proved that minimizing the loss in CL-STE method using gradient descent via a straight-through-estimator updates the neural network’s weights in the direction that the binarized outputs satisfy the logical constraints. Our experimental results on CL-STE show that a neural network can be trained better with less data and fewer labels when semantic constraints are given through CL-STE. Improvements of accuracy have been observed on fully-, semi-, and un-supervised learning tasks over different types of neural networks including multi-layer perceptron (MLP), convolutional neural networks (CNN), graph neural networks (GNN), and Transformers. More importantly, leveraging GPUs and batch training, CL-STE method scales significantly better on a big range of existing benchmark problems compared to state-of-the-art neuro-symbolic methods that use heavy symbolic computation as a blackbox for computing gradients.

When applying CL-STE to Transformers, we found that Transformers can be extended with recurrence to enhance its power for multi-step reasoning and can be a viable approach to learning to solve Constraint Satisfaction Problems (CSPs) in an end-to-end manner. We designed `Recurrent Transformer` (Yang *et al.*, 2023) and showed

that it has clear advantages over state-of-the-art methods such as Graph Neural Networks (Palm *et al.*, 2018), SATNet (Wang *et al.*, 2019), and some neuro-symbolic models (Yang *et al.*, 2020; Bai *et al.*, 2021). With the ability of Transformer to handle visual input, Recurrent Transformer can straightforwardly be applied to visual constraint reasoning problems while successfully addressing the symbol grounding problem (Chang *et al.*, 2020; Topan *et al.*, 2021). We also designed a variant of CL-STE, which is restricted to cardinality constraints only but is even more computationally efficient than CL-STE. We showed how to leverage deductive knowledge of discrete constraints in the Transformer’s inductive learning to achieve sample-efficient learning and semi-supervised learning for CSPs.

Recently, LLMs have shown wide success on many downstream tasks, demonstrating general reasoning capability on diverse tasks without being retrained. However, when we restrict our attention to individual NLP reasoning benchmarks, they usually do not perform as well as state-of-the-art models despite various efforts to improve accuracy through prompt engineering (Wei *et al.*, 2022; Zhou *et al.*, 2022). As training LLMs is not capable, we limited our attention to inference with LLMs and designed a dual-process neuro-symbolic reasoning system LLM+ASP. We found that the rich semantic knowledge that LLMs possess makes them effective general-purpose few-shot semantic parsers that can convert linguistically variable natural language sentences into atomic facts that serve as input to logic programs. We also found that the fully declarative nature of answer set programs (Lifschitz, 2008a; Brewka *et al.*, 2011b) makes them a good pair with the LLM semantic parsers, providing interpretable and explainable reasoning on the parsed result of the LLMs using background knowledge. LLM+ASP works across multiple QA tasks without retraining for individual tasks. It requires only a few examples to direct an LLM (i.e., GPT-3) to tune to an individual task, along with ASP knowledge modules that can be reused over multiple tasks. We demonstrated that this method achieves state-of-the-art

performance on several NLP benchmarks, such as bAbI (Weston *et al.*, 2016), StepGame (Shi *et al.*, 2022), CLUTRR (Sinha *et al.*, 2019), and gSCAN (Ruis *et al.*, 2020), and also handles robot planning tasks that an LLM alone fails to solve.

The dissertation is organized as follows. Chapter 2 examines related works. Chapter 3 presents a neuro-symbolic formalism `NeurASP`, which integrates neural networks with answer set programs. Chapter 4 describes a semantic regularization method `CL-STE`. Chapter 5 introduces `Recurrent Transformer` and a semantic loss stemmed from `CL-STE`. In Chapter 6, we present `LLM+ASP`, which augments LLMs with symbolic reasoning in ASP for textual question-answering. Chapter 7 concludes.

PREVIEW

BACKGROUND

2.1 Neuro-Symbolic Methods

Recent years have observed the rising interest in combining neural and symbolic systems (Marcus, 2018; Lamb *et al.*, 2020; Sarker *et al.*, 2021).

Xu *et al.* (2018) proposed a semantic loss function to bridge neural network (NN) output and logical constraints. The method treats NN output as probabilities and computes semantic loss as the negative logarithm of the probability to generate a state satisfying the logical constraints. Their experiments show that the encoded semantic loss function guides the learner to achieve state-of-the-art results on supervised and semi-supervised learning on multi-class classification. For the efficient computation of a loss function, they encode logical constraints in Sentential Decision Diagram (SDD) (Darwiche, 2011). However, generating SDDs is computationally expensive for most practical tasks.

Several neuro-symbolic formalisms, such as Logic Tensor Network (Serafini and Garcez, 2016), DeepProbLog (Manhaeve *et al.*, 2018), NeuroLog (Tsamoura *et al.*, 2021), DeepStochLog (Winters *et al.*, 2021), and fuzzy logic regularizer (Roychowdhury *et al.*, 2021), have been proposed to enforce logical constraints in neural network training by appending a logic layer to an existing neural network. Many of them treat the logic component as a blackbox module (Pogancic *et al.*, 2020) to neural networks. Since discrete logical inference cannot be in general captured via a differentiable function, they use relaxation to fuzzy logic or weighted models or probability. Our work NeurASP (Yang *et al.*, 2020) is also of this kind where the logic layer is built upon the ASP solver CLINGO (Lifschitz, 2008b; Brewka *et al.*, 2011a; Calimeri *et al.*, 2020). While this approach provides a

systematic representation of constraints, the symbolic computation is often a bottleneck.

Another approach is to embed logic rules in neural networks by representing logical connectives by mathematical operations and allowing the value of an atom to be a real number. For example, Neural Theorem Prover (NTP) (Rocktäschel and Riedel, 2017) adopts the idea of dynamic neural module networks (Andreas *et al.*, 2016) to embed logic conjunction and disjunction in and/or-module networks. A proof-tree like end-to-end differentiable neural network is then constructed using Prolog’s backward chaining algorithm with these modules. Another method that also constructs a proof-tree like neural network is TensorLog (Cohen *et al.*, 2018), which uses matrix multiplication to simulate belief propagation that is tractable under the restriction that each rule is negation-free and can be transformed into a polytree.

Other works train neural networks for learning satisfiability, such as (Wang *et al.*, 2019; Selsam *et al.*, 2019). SATNet (Wang *et al.*, 2019) builds on a line of research exploring SDP relaxations as a tool for solving MAXSAT, which produces tighter approximation guarantees than standard linear programming relaxation. Remarkably, their method learns to solve Sudoku puzzles without any hand-coded knowledge. On the other hand, our experiments show that SATNet does not extrapolate well — it does not work well for reasonably harder Sudoku instances than those that it is trained on.

Graph Neural Networks (GNNs) (Kipf and Welling, 2017; Battaglia *et al.*, 2018; Lamb *et al.*, 2020) have been widely applied. Since a graph can encode objects and relations between objects, by learning message functions between the nodes, one can perform certain relational reasoning over the objects. For example, ExpressGNN (Zhang *et al.*, 2019) constructs a graph neural network to simulate variational inference in Markov Logic Network. Recurrent Relational Network (RRN) (Palm *et al.*, 2018) is a state-of-the-art GNN for multi-step relational reasoning, achieving 96.6% accuracy for Sudoku problems. GNNs use message-passing to propagate logical constraints in neural networks, but they do not

have the mechanism to specify the logical constraints directly.

Neuro-Symbolic Concept Learner (Mao *et al.*, 2019) separates between visual perception and symbolic reasoning. It shows the data-efficiency by using only 10% of the training data and achieving the state-of-the-art 98% accuracy on CLEVR dataset. Such kind of dual-system models achieved new state-of-the-art results in visual QA (Goldman *et al.*, 2018; Sampat and Lee, 2018; Yi *et al.*, 2019; Chen *et al.*, 2020; Ding *et al.*, 2021). In the case of textual problems, to improve LLMs to generate more consistent and coherent sentences, Nye *et al.* (2021) suggest that generation be decomposed into two parts: candidate sentence generation by an LLM (system 1 thinking) and a logical pruning process (system 2 thinking) implemented via a separate symbolic module. They show that this neuro-symbolic, dual-process model requires fewer data to learn and achieves higher accuracy and better generalization.

2.2 Answer Set Programs

Answer Set Programming (ASP) (Lifschitz *et al.*, 2001), based on the stable model semantics (Gelfond and Lifschitz, 1988), is a widely-used Knowledge Representation (KR) framework that facilitates elegant and efficient representations for many problem domains that require complex reasoning.

We assume a first-order signature σ that contains no function constants of positive arity, which yields finitely many Herbrand interpretations. The syntax of formulas is defined the same as in the standard first-order logic. We say that a formula is *negative* if every occurrence of every atom in this formula is in the scope of negation.

In this dissertation, for simplicity, we mainly consider a *rule* of the form

$$A \leftarrow B \wedge N \tag{2.1}$$

where A is a disjunction of atoms, B is a conjunction of atoms, and N is a negative formula

constructed from atoms using conjunction, disjunction and negation. We identify rule (2.1) with formula $B \wedge N \rightarrow A$. We often use comma for conjunction, semi-colon for disjunction, *not* for negation, as widely used in the literature on logic programming. For example, N could be

$$\neg B_{m+1} \wedge \dots \wedge \neg B_n \wedge \neg \neg B_{n+1} \wedge \dots \wedge \neg \neg B_p,$$

which can be also written as

$$\text{not } B_{m+1}, \dots, \text{not } B_n, \text{not not } B_{n+1}, \dots, \text{not not } B_p.$$

We write $\{A_1\}^{\text{ch}} \leftarrow \text{Body}$, where A_1 is an atom, to denote the rule $A_1 \leftarrow \text{Body} \wedge \neg \neg A_1$. This expression is called a “choice rule” in ASP.

If the head of a rule (A in (2.1)) is $\perp$, we often omit it and call such a rule *constraint*.

A logic program under the stable model semantics (a.k.a. *answer set program*) is a finite conjunction of rules. A logic program is called *ground* if it contains no variables.

We say that an Herbrand interpretation I is a *model* of a ground program Π if I satisfies all implications (2.1) in Π (as in classical logic). Such models can be divided into two groups: “stable” and “non-stable” models, which are distinguished as follows. The *reduct* of Π relative to I , denoted Π^I , consists of “ $A \leftarrow B$ ” for all rules (2.1) in Π such that $I \models N$ ($I \models N$ denotes “ I satisfies N ” in classical logic). The Herbrand interpretation I is called a (*deterministic*) *stable model* of Π if I is a minimal Herbrand model of Π^I . (Minimality is understood in terms of set inclusion. We identify an Herbrand interpretation with the set of atoms that are true in it.)

The definition is extended to any non-ground program Π by identifying it with $gr_\sigma[\Pi]$, the ground program obtained from Π by replacing every variable with every ground term of the signature σ .

A *weak constraint* (Buccafurri *et al.*, 2000; Calimeri *et al.*, 2012) has the form

$$:\sim F \quad [Weight @ Level]$$

where F is a conjunction of literals, $Weight$ is a real number, and $Level$ is a nonnegative integer.

Let Π be a program $\Pi_1 \cup \Pi_2$, where Π_1 is an answer set program that does not contain weak constraints, and Π_2 is a set of ground weak constraints. We call I a *stable model* of Π if it is a stable model of Π_1 . For every stable model I of Π and any nonnegative integer l , the *penalty* of I at level l , denoted by $Penalty_{\Pi}(I, l)$, is defined as

$$\sum_{\substack{I \sim F[w@l] \in \Pi_2, \\ I \models F}} w.$$

For any two stable models I and I' of Π , we say I is *dominated* by I' if

- there is some nonnegative integer l such that $Penalty_{\Pi}(I', l) < Penalty_{\Pi}(I, l)$ and
- for all integers $k > l$, $Penalty_{\Pi}(I', k) = Penalty_{\Pi}(I, k)$.

A stable model of Π is called *optimal* if it is not dominated by another stable model of Π .